

Lecture 4

Resampling Methods

4.1

Contents

1	Introduction	1
2	Validation Set Approach	2
3	Leave-One-Out-Cross-Validation (LOOCV)	3
4	<i>k</i>-Fold Cross-Validation	4
5	The Bootstrap	5
6	Regression Analysis of the Belgium Wage Data Set	7
6.1	Cross Validation	13
6.2	Bootstrap Non-Parametric Evaluation	18
6.3	Effect Sizes	20

Some figures in this presentation are taken from 'An Introduction to Statistical Learning, with applications in R' (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani.

Most of the material is extracted from the slides of the MOOC course by Hastie and Tibshirani, available at <https://www.r-bloggers.com/in-depth-introduction-to-machine-learning-in-15-hours-of-expert-videos/>.

4.2

1 Introduction

Resampling methods involve repeatedly drawing samples from a training set and refitting a model of interest on each sample in order to obtain additional information about the fitted model, e.g. with regard to the point estimate of the parameters in the model.

Resampling methods are typically used when only the training data set is available for estimation (in that case, inferential conclusions rests on model assumptions and on asymptotic results, e.g. the central limit theorem).

4.3

Model assessment

Methods for evaluating a model performance.

Model selection

Methods for selecting the most appropriate flexibility level.

Cross validation

$MSE_{training}$ vs MSE_{test}

A good statistical learning method has low MSE_{test}

4.4

Approximation of the MSE_{test}

In absence of a very large designated test set that can be used to directly estimate the test error rate we can have recourse to:

- mathematical adjustment of $MSE_{training}$, see Ch. 6 JWHT (Akaike, BIC, Mallow's C_p and Adjusted R^2)
- an estimate of MSE_{test} starting from the training data set

□□□□□...□□□■...■

□...□, a subset of the training data set is used to estimate $\hat{f}$

■...■, another part of the training data set is *hold out* and used to estimate MSE_{test}

4.5

Solutions

- Validation set approach
- Leave One Out Cross Validation (LOOCV)
- k -Fold Cross Validation

In order to give a graphical representation of the methods, we assume that the training data set is made up by 20 units.

4.6

2 Validation Set Approach

Let us consider a random permutation of the indices $1, 2, \dots, n$ of the statistical units $x_1, x_2, \dots, x_n$ that build the data set

1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20

for example

5,17,8,15,16,7,19,11,14,9,20,12,1,10,18,3,2,4,6,13

The set of permuted data is divided into two parts, usually having equal length. In our example the two parts are identified by the two sets of indices

5,17,8,15,16,7,19,11,14,9

20,12,1,10,18,3,2,4,6,13

- the first set of observations will be used as *training set* and
- the second one as *validation set* or *hold-out set*

4.7

5,17,8,15,16,7,19,11,14,9

training data set

↓
 $\hat{f}$

model estimation

20,12,1,10,18,3,2,4,6,13

validation data set

↓
 $y - \hat{f}(x)$

MSE_{test} estimation

4.8

Drawbacks of the validation set approach

- MSE_{test} depends on which observations are included in the training data set and which observations are included in the validation set; therefore the validation estimate of MSE_{test} can be highly variable.
- Since the performance of a statistical learning method is a direct function of the number of observations used to estimate $\hat{f}$, we can observe an *overestimate* of MSE_{test} , when compared with the situation where $\hat{f}$ is obtained by considering the whole data set.

4.9

3 Leave-One-Out-Cross-Validation (LOOCV)

The following procedure is iterated n times, starting from the observations $x_1, x_2, \dots, x_n$ that build the data set.

Consider the training data set obtained from the initial data set by dropping the i th observation, estimate f and obtain with reference to the i th observation

$$MSE_i = (y_i - \hat{f}(x_i))^2.$$

4.10

Therefore, with reference to the set of indices that identify the statistical units

1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20

we consider the following training data set, the 'test data set' given by the i th observation, and the n 'partial' estimates MSE_i of MSE_{test}

2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20	1	$\rightarrow MSE_1 = (y_1 - \hat{f}(x_1))^2$
1,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20	2	$\rightarrow MSE_2 = (y_2 - \hat{f}(x_2))^2$
1,2,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20	3	$\rightarrow MSE_3 = (y_3 - \hat{f}(x_3))^2$
1,2,3,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20	4	$\rightarrow MSE_4 = (y_4 - \hat{f}(x_4))^2$
1,2,3,4,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20	5	$\rightarrow MSE_5 = (y_5 - \hat{f}(x_5))^2$
1,2,3,4,5,7,8,9,10,11,12,13,14,15,16,17,18,19,20	6	$\rightarrow MSE_6 = (y_6 - \hat{f}(x_6))^2$
1,2,3,4,5,6,8,9,10,11,12,13,14,15,16,17,18,19,20	7	$\rightarrow MSE_7 = (y_7 - \hat{f}(x_7))^2$
1,2,3,4,5,6,7,9,10,11,12,13,14,15,16,17,18,19,20	8	$\rightarrow MSE_8 = (y_8 - \hat{f}(x_8))^2$
1,2,3,4,5,6,7,8,10,11,12,13,14,15,16,17,18,19,20	9	$\rightarrow MSE_9 = (y_9 - \hat{f}(x_9))^2$
1,2,3,4,5,6,7,8,9,11,12,13,14,15,16,17,18,19,20	10	$\rightarrow MSE_{10} = (y_{10} - \hat{f}(x_{10}))^2$
1,2,3,4,5,6,7,8,9,10,12,13,14,15,16,17,18,19,20	11	$\rightarrow MSE_{11} = (y_{11} - \hat{f}(x_{11}))^2$
1,2,3,4,5,6,7,8,9,10,11,13,14,15,16,17,18,19,20	12	$\rightarrow MSE_{12} = (y_{12} - \hat{f}(x_{12}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,14,15,16,17,18,19,20	13	$\rightarrow MSE_{13} = (y_{13} - \hat{f}(x_{13}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,15,16,17,18,19,20	14	$\rightarrow MSE_{14} = (y_{14} - \hat{f}(x_{14}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,16,17,18,19,20	15	$\rightarrow MSE_{15} = (y_{15} - \hat{f}(x_{15}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,17,18,19,20	16	$\rightarrow MSE_{16} = (y_{16} - \hat{f}(x_{16}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,18,19,20	17	$\rightarrow MSE_{17} = (y_{17} - \hat{f}(x_{17}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,19,20	18	$\rightarrow MSE_{18} = (y_{18} - \hat{f}(x_{18}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,20	19	$\rightarrow MSE_{19} = (y_{19} - \hat{f}(x_{19}))^2$
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19	20	$\rightarrow MSE_{20} = (y_{20} - \hat{f}(x_{20}))^2$

4.11

The estimate of MSE_{test} is given by

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n MSE_i.$$

4.12

Advantages of the LOOCV

The two drawbacks of the validation set approach are overcome

- The estimate $\hat{f}$ of f is made by using $n - 1$ observations. The LOOCV approach tends not to overestimate MSE_{test} .
- There is no randomness in the training/validation set splits.

4.13

Drawbacks of LOOCV

- The model has to be fit n times. This can be very time consuming if n is large, and if each individual model is slow to fit (computational intensive/heavy problems). With least squares linear or polynomial regression, an amazing shortcut makes the cost of LOOCV the same as that of a single model fit! The following formula holds

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2$$

where $\hat{y}_i$ is the OLS fitted value of y_i and h_i is the leverage, that is the i th element on the main diagonal of the hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'$.

4.14

4 k -Fold Cross-Validation

Let us consider a random permutation of the indices of the observations $x_1, x_2, \dots, x_n$ that build the data set

1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20

for example

20,3,4,19,5,8,15,11,1,10,13,2,16,7,17,12,14,6,9,18

The set of the permuted data is divided into k groups, or *folds*, with approximately equal size¹ (usually $k = 5$ or $k = 10$), identified, in our illustration, by $k = 5$ sets of indices $Ind_1, \dots, Ind_5$

20,3,4,19 5,8,15,11 1,10,13,2 16,7,17,12 14,6,9,18

4.15

The following procedure is repeated k times.

Consider as training data set the union of the k groups just defined, leaving out the i th ($i = 1, \dots, k$), estimate f and obtain the estimate

$$MSE_i = \frac{k}{n} \sum_{j \in Ind_i} (y_j - \hat{f}(x_j))^2$$

referred to the i th group (that was excluded from the training data set).

4.16

We consider the following training data sets, the 'test data set' given by the i th excluded group, and the k 'partial' estimates MSE_i of MSE_{test}

5,8,15,11	1,10,13,2	16,7,17,12	14,6,9,18	20,3,4,19	→	$MSE_1 = [(y_{20} - \hat{f}(x_{20}))^2 + \dots + (y_{19} - \hat{f}(x_{19}))^2] / 4$
20,3,4,19	1,10,13,2	16,7,17,12	14,6,9,18	5,8,15,11	→	$MSE_2 = [(y_5 - \hat{f}(x_5))^2 + \dots + (y_{11} - \hat{f}(x_{11}))^2] / 4$
20,3,4,19	5,8,15,11	16,7,17,12	14,6,9,18	1,10,13,2	→	$MSE_3 = [(y_1 - \hat{f}(x_1))^2 + \dots + (y_2 - \hat{f}(x_2))^2] / 4$
20,3,4,19	5,8,15,11	1,10,13,2	14,6,9,18	16,7,17,12	→	$MSE_4 = [(y_{16} - \hat{f}(x_{16}))^2 + \dots + (y_{12} - \hat{f}(x_{12}))^2] / 4$
20,3,4,19	5,8,15,11	1,10,13,2	16,7,17,12	14,6,9,18	→	$MSE_5 = [(y_{14} - \hat{f}(x_{14}))^2 + \dots + (y_{18} - \hat{f}(x_{18}))^2] / 4$

4.17

The estimate of MSE_{test} is given by

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i.$$

The LOOCV corresponds to the k -fold CV with $k = n$.

4.18

Advantages of the k -Fold Cross Validation

- The fit procedure is repeated $k < n$ times, and calculations can be more feasible with respect to LOOCV in presence of statistical learning methods that are computational intensive.
- k -fold CV often gives more accurate estimates of the test error rate than does LOOCV. This has to do with a bias-variance trade-off.
 - With reference to the bias component of the estimate of MSE_{test} the following preference ordering holds

$$LOOCV \succ k\text{-Fold Cross Validation} \succ \text{Validation set}$$

Namely, usually it holds that

$$MSE.bias_{LOOCV} < MSE.bias_{k\text{-Fold Cross Validation}} \ll MSE.bias_{\text{Validation set}}$$

- With reference to the variance component in the estimate of MSE_{test} the following preference ordering holds

$$k\text{-Fold Cross Validation} \succ LOOCV$$

we have

$$Var_{k\text{-Fold Cross Validation}} < Var_{LOOCV}$$

Namely, the LOOCV estimate of MSE_{test} is obtained as average of n partial estimates obtained from n almost identical training data set (only one observation changes); the n partial estimates are, then, characterized by high (positive) correlation, with the consequence of increasing the variance of the estimate². The partial estimates of MSE_{test} obtained with the k -Fold Cross Validation are referred to k training data set with smaller overlaps and thus a higher variability. Therefore, the partial estimates have lower correlation than those obtained with the LOOCV.

Empirical results by JWHT show that $k = 5$ or $k = 10$ groups yield test error rate estimates that suffer neither from excessively high bias nor from very high variance.

4.19

4.20

4.21

4.22

5 The Bootstrap

The bootstrap is a widely applicable and extremely powerful statistical tool that can be used to quantify the uncertainty associated with a given estimator or statistical learning method.

It can be used to find the empirical distribution of the estimators of the parameters that are present in a model.

¹For the sake of simplicity in deriving relationships, we now assume that all folds have the same size.

²Remember that

$$Var(\bar{X}) = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n Cov(X_i, X_j) = \frac{1}{n^2} \sum_{i=1}^n Var(X_i) + \frac{1}{n^2} \sum_{i \neq j} Cov(X_i, X_j)$$

Inference on parameter estimates is, usually, based on the assumption made on the error component.

Consider, for example, the t test referred to the estimates of the coefficients in a linear regression model; the Student t distribution holds, irrespective of the sample size, only if the assumption of Normal distribution of the errors holds (only if we are in presence of a linear Gaussian classical regression model).

Sometimes it is necessary to have recourse to the central limit theorem, by concluding that estimators are asymptotically Normally distributed.

But what does *asymptotically* mean?

The Bootstrap procedure consists in extracting $B \gg 1$ samples from the n observations; each sample has size n and its components are drawn with replication from the set of the initial data.

Parameters are estimated for each of the B samples and, starting from the B estimates one can build histograms or kernel estimates of the density function, and compute mean, standard deviation and empirical confidence intervals.

The standard deviation can be used as estimate of the standard error of the parameter estimators.

Figure extracted from James, Witten, Hastie, Tibshirani (2013) An Introduction to Statistical Learning, Springer

4.23

4.24

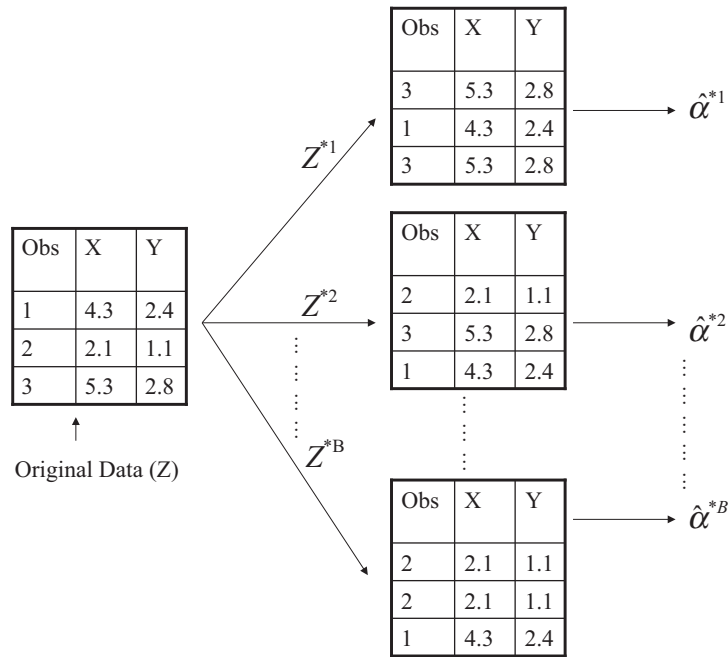


FIGURE 5.11. A graphical illustration of the bootstrap approach on a small sample containing $n = 3$ observations. Each bootstrap data set contains n observations, sampled with replacement from the original data set. Each bootstrap data set is used to obtain an estimate of α .

4.25

We have the B estimates

$$\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}.$$

referred to a generic parameter α .

Their average

$$\frac{1}{B} \sum_{r=1}^B \hat{\alpha}^{*r}$$

corresponds to the bootstrap estimate of α .

The standard deviation of the B estimates

$$\sqrt{\frac{1}{B-1} \sum_{r=1}^B \left(\hat{\alpha}^{*r} - \frac{1}{B} \sum_{r=1}^B \hat{\alpha}^{*r} \right)^2}$$

is an estimate of the standard error of the estimator used for α .

4.26

6 Regression Analysis of the Belgium Wage Data Set

Let us consider again the analysis of the Belgium wage data set

```
indwages <- read.table("bwages.dat", header = TRUE)
```

In order to perform the analysis by using directly categorical variables, the dummy variables MALE and EDUC are transformed in the factors sex and educf, respectively.

```
indwages <- transform(indwages, sex = factor(MALE, levels = 0:1,
  labels = c("f", "m")))
names(indwages)
## [1] "WAGE" "LNWAGE" "EDUC" "EXPER" "LNEXPER"
## [6] "LNEDUC" "MALE" "sex"
indwages <- transform(indwages, educf = factor(EDUC))
```

4.27

As initial specification we study WAGE as a function of sex, educf and EXPER

```
output1 <- lm(WAGE ~ sex + educf + EXPER, data = indwages)
```

summary can be used to obtain the model output

```
summary(output1)
```

Remember that it is useless to interpret coefficients before having assessed if the model is valid.

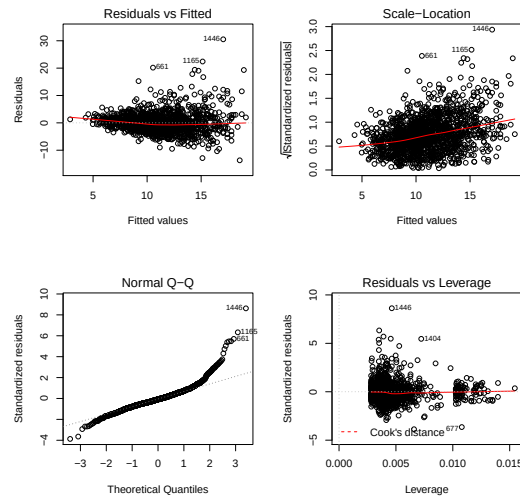
4.28

We first check for the presence of multicollinearity.

```
library(car)
## Loading required package: carData
vif(output1)
##          GVIF Df GVIF^(1/(2*Df))
## sex      1.057576 1      1.028385
## educf    1.142048 4      1.016741
## EXPER    1.123141 1      1.059783
```

No multicollinearity issues emerge; so we can analyze the plot of residuals.

```
layout(matrix(1:4, 2, 2))
plot(output1)
```



Heteroscedasticity and deviation from normality are present. The presence of heteroscedasticity is also confirmed by the Breusch Pagan test.

```
library(lmtest)
bptest(output1)

##
## studentized Breusch-Pagan test
##
## data: output1
## BP = 99.164, df = 6, p-value < 2.2e-16
```

Following Verbeek, we can try to adjust for heteroscedasticity by using the logarithm of WAGE, LNWAGE, as dependent variable, and using LNEXPER, the logarithm of (EXPER + 1) instead of EXPER as predictor.

The two following specifications of the model are equivalent.

```
output2 <- lm(log(WAGE) ~ sex + educf + log(EXPER + 1), data = indwages)
output2 <- lm(LNWAGE ~ sex + educf + LNEXPER, data = indwages)
```

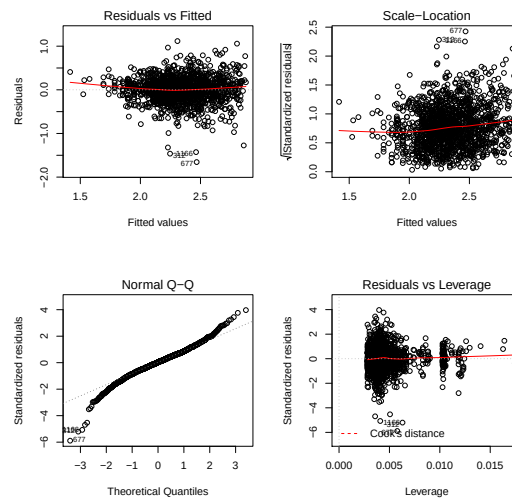
The VIF analysis confirms that there is no multicollinearity, also with the logarithmic transformation of EXPER

```
vif(output2)

##              GVIF Df GVIF^(1/(2*Df))
## sex          1.055532 1          1.027391
## educf        1.120396 4          1.014312
## LNEXPER      1.096974 1          1.047365
```

We can proceed to produce the residuals plot.

```
layout(matrix(1:4, 2, 2))
plot(output2)
```



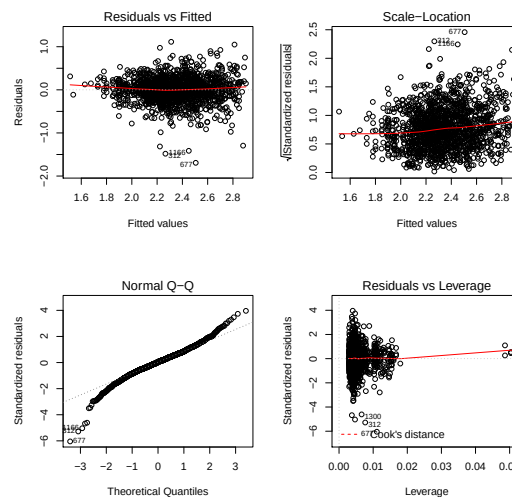
4.34

Heteroscedasticity remains, but is less pronounced than with the initial model specification.

We update the model specification by including also the square of `LNEXPER` (observe the use of the `as.is` function `I()`), and plot residuals.

```
output3 <- lm(LNWAGE ~ sex + educf + LNEXPER + I(LNEXPER^2),
              data = indwages)
layout(matrix(1:4, 2, 2))
plot(output3)
```

4.35



4.36

We can obtain studentized residuals

```
rstudent(output3)
```

and check which values are possible outliers, that is observations with absolute studentized residuals larger than 3.

```
which(abs(rstudent(output3)) > 3)
```

We can obtain leverages, in order to detect possible leverage data

```
hatvalues(output3)
```

and check which values are leverage data.

```
which(hatvalues(output3) > 2 * length(output3$coef)/length(output3$res))
```

In order to find influential data we have to check which observations satisfy both conditions regarding studentized residuals and leverages.

```
influential <- which((abs(rstudent(output3)) > 3) & (hatvalues(output3) > 2 * length(output3$coef)/length(output3$res)))
```

If possible, information given by influential as well as by outlier subjects should be carefully checked in order to correct possible errors made in retrieving data.

```
indwages[influential, ]
##      WAGE      LNWAGE EDUC  EXPER  LNEXPER  LNEDUC MALE sex educf
## 677 2.253577 0.8125189    3    47 3.871201 1.098612    0  f     3

summary(indwages)
##      WAGE      LNWAGE      EDUC      EXPER      LNEXPER      LNEDUC      MALE      sex      educf
##  Min.   : 2.191   Min.   :0.7843   Min.   :1.000   Min.   : 0.00   Min.   :0.000   Min.   :0.000   Min.   :0.0000   f:579   1: 99
## 1st Qu.: 8.113   1st Qu.:2.0935   1st Qu.:3.000   1st Qu.: 9.00   1st Qu.:2.303   1st Qu.:1.099   1st Qu.:0.0000   m:893   2:265
## Median :10.127   Median :2.3152   Median :3.000   Median :16.50   Median :2.862   Median :1.099   Median :1.0000               3:420
## Mean   :11.051   Mean   :2.3344   Mean   :3.378   Mean   :17.22   Mean   :2.691   Mean   :1.137   Mean   :0.6067               4:356
## 3rd Qu.:12.755   3rd Qu.:2.5460   3rd Qu.:4.000   3rd Qu.:24.00   3rd Qu.:3.219   3rd Qu.:1.386   3rd Qu.:1.0000               5:332
## Max.   :47.576   Max.   :3.8623   Max.   :5.000   Max.   :47.00   Max.   :3.871   Max.   :1.609   Max.   :1.0000
```

Also the Cook distance gives information about possible influential data.

```
cooks.distance(output3)
which(cooks.distance(output3) > 1)
##      named integer(0)
```

We can plot studentized residuals against hat values and identify single points by clicking on the graph.

```
plot(rstudent(output3) ~ hatvalues(output3))
identify(rstudent(output3) ~ hatvalues(output3))
```

Finally, we estimate the model by excluding influential observations.

```
output4 <- lm(LNWAGE ~ sex + educf + LNEXPER + I(LNEXPER^2),
  data = indwages, subset = (1:nrow(indwages))[-influential])
```

```
which((abs(rstudent(output4)) > 3) & (hatvalues(output4) > 2 * length(output4$coef)/length(output3$res)))
##      named integer(0)
```

Table 1 shows a fancy output to compare the two lm outputs output3 and output4. It can be obtained with the function stargazer available in the package stargazer.

```
summary(output3)
summary(output4)
library(stargazer)
stargazer(output3, output4, type = "text", report = "vcp*")
stargazer(output3, output4, report = "vcp*", label = "table:output3output4")
```

By having removed a single influential statistical unit, both LNEXPER and its square become significant (relying on the normal distribution of errors or on asymptotic results).

The model with the squared LNEXPER can be re-written, in a more efficient way, by using the function poly.

```
output5 <- lm(LNWAGE ~ sex + educf + poly(LNEXPER, 2, raw = TRUE),
  data = indwages, subset = (1:nrow(indwages))[-influential])
```

A comparison of the two model coefficients shows that they refer to the same model.

Table 1:

	<i>Dependent variable:</i>	
	LNWAGE	
	(1)	(2)
sexm	0.116 p = 0.000***	0.112 p = 0.000***
educf2	0.148 p = 0.00002***	0.149 p = 0.00001***
educf3	0.312 p = 0.000***	0.319 p = 0.000***
educf4	0.483 p = 0.000***	0.486 p = 0.000***
educf5	0.646 p = 0.000***	0.649 p = 0.000***
LNEXPER	0.134 p = 0.013**	0.114 p = 0.033**
I(LNEXPER^2)	0.021 p = 0.068*	0.026 p = 0.023**
Constant	1.364 p = 0.000***	1.380 p = 0.000***
Observations	1,472	1,471
R ²	0.399	0.407
Adjusted R ²	0.396	0.404
Residual Std. Error	0.282 (df = 1464)	0.278 (df = 1463)
F Statistic	138.820*** (df = 7; 1464)	143.345*** (df = 7; 1463)

Note:

*p<0.1; **p<0.05; ***p<0.01

```
coef(output4)
##      (Intercept)      sexm      educf2      educf3
##      1.37966965    0.11224023    0.14936568    0.31880404
##      educf4      educf5      LNEXPER I (LNEXPER^2)
##      0.48600075    0.64900984    0.11374627    0.02573268
```

4.47

```
coef(output5)
##      (Intercept)      sexm
##      1.37966965    0.11224023
##      educf2      educf3
##      0.14936568    0.31880404
##      educf4      educf5
##      0.48600075    0.64900984
## poly(LNEXPER, 2, raw = TRUE)1 poly(LNEXPER, 2, raw = TRUE)2
##      0.11374627    0.02573268
```

4.48

The formulation considered in output5 shows clear advantages regarding the check for multicollinearity and also to perform analysis of variance tests.

In both cases it is considered the linear subspace defined by the whole set of levels of a categorical variable or by the whole set of powers associated to a single predictor, instead of single levels or single powers.

```
vif(output4)
##      GVIF Df GVIF^(1/(2*Df))
## sex      1.059035 1      1.029094
## educf    1.151084 4      1.017744
## LNEXPER  28.506837 1      5.339179
## I (LNEXPER^2) 28.974041 1      5.382754
```

```
vif(output5)
##      GVIF Df GVIF^(1/(2*Df))
## sex      1.059035 1      1.029094
## educf    1.151084 4      1.017744
## poly(LNEXPER, 2, raw = TRUE) 1.134519 2      1.032055
```

4.49

With anova we can check, under the hypothesis of normality of the errors, if a regressor is significant, and the check is correctly made, in an easy way, with reference to the whole set of levels of a categorical variable or to the whole set of powers associated to a single predictor (when poly is used), instead of single levels or single powers.

```
output5bis <- lm(LNWAGE ~ sex + poly(LNEXPER, 2, raw = TRUE),
  data = indwages, subset = (1:nrow(indwages))[-influential])
output5ter <- lm(LNWAGE ~ sex + educf, data = indwages, subset = (1:nrow(indwages))[-influential])
```

4.50

```
anova(output5bis, output5)
## Analysis of Variance Table
##
## Model 1: LNWAGE ~ sex + poly(LNEXPER, 2, raw = TRUE)
## Model 2: LNWAGE ~ sex + educf + poly(LNEXPER, 2, raw = TRUE)
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1     1467 165.88
## 2     1463 113.31  4     52.569 169.69 < 2.2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(output5ter, output5)
## Analysis of Variance Table
##
## Model 1: LNWAGE ~ sex + educf
## Model 2: LNWAGE ~ sex + educf + poly(LNEXPER, 2, raw = TRUE)
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1     1465 152.40
## 2     1463 113.31  2     39.095 252.4 < 2.2e-16 ***
## ---
## Signif. codes:
## 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.51

We can perform tests on the residuals distribution with the function `fit.cont` available in the package `rriskDistributions`, in order to check if the normality assumption is met.

```
library(rriskDistributions)
fit.cont(output5$res)
```

Residuals are not normally distributed. Therefore, we need bootstrap confidence intervals in order to evaluate the significance of model coefficients.

4.52

In order to simplify notation, we re-define the dataframe `indwages` by dropping influential observations.

```
indwages <- indwages[-influential, ]
```

4.53

The following re-writing highlights separate contributions of single powers of `LNEXPER` in explaining the outcome.

```
output5 <- lm(LNWAGE ~ sex + educf + poly(LNEXPER, 2), data = indwages)
```

A linear rotation of the polynomial predictors is made, in order to have orthogonal predictors all with the same variance.

```
cov(poly(indwages$LNEXPER, 2))
##           1           2
## 1  6.802721e-04 -2.512370e-19
## 2 -2.512370e-19  6.802721e-04
```

We observe that it is more difficult to establish the expected change in the wage following to an additional year of experience. The original expression `poly(LNEXPER, 2, raw=TRUE)` makes this task easier.

4.54

6.1 Cross Validation

Before assessing non-parametric bootstrap evaluation, we try to find the optimal order of the polynomial in `LNEXPER` from a predictive point of view.

```
output6 <- lm(LNWAGE ~ sex + educf + poly(LNEXPER, 4), data = indwages)
output7 <- lm(LNWAGE ~ sex + educf + poly(LNEXPER, 3), data = indwages)
```

4.55

```
lmtest::coeftest(output6)
##
## t test of coefficients:
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.888080   0.030773  61.3559 < 2.2e-16 ***
## sexm          0.112017   0.015282   7.3299 3.797e-13 ***
## educf2        0.145908   0.033062   4.4132 1.094e-05 ***
```

```
## educf3          0.315732    0.031911    9.8942 < 2.2e-16 ***
## educf4          0.484097    0.032936   14.6982 < 2.2e-16 ***
## educf5          0.646757    0.033024   19.5843 < 2.2e-16 ***
## poly(LNEXPER, 4)1  6.548059    0.291745   22.4444 < 2.2e-16 ***
## poly(LNEXPER, 4)2  0.647416    0.282808    2.2892    0.02221 *
## poly(LNEXPER, 4)3 -0.490911    0.279009   -1.7595    0.07871 .
## poly(LNEXPER, 4)4  0.161892    0.278293    0.5817    0.56084
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.56

```
coeftest(output7)
##
## t test of coefficients:
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.888465   0.030759  61.3965 < 2.2e-16 ***
## sexm           0.111984   0.015279   7.3295 3.808e-13 ***
## educf2         0.145470   0.033046    4.4020 1.150e-05 ***
## educf3         0.315330   0.031896    9.8862 < 2.2e-16 ***
## educf4         0.483848   0.032925   14.6952 < 2.2e-16 ***
## educf5         0.646264   0.033006   19.5803 < 2.2e-16 ***
## poly(LNEXPER, 3)1  6.547518   0.291678   22.4478 < 2.2e-16 ***
## poly(LNEXPER, 3)2  0.647285   0.282743    2.2893    0.02220 *
## poly(LNEXPER, 3)3 -0.491240   0.278945   -1.7611    0.07844 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.57

Which power of $\text{poly}(\text{LNEXPER})$ is to be considered? 3 seems to be a better choice than 4. Though its interpretation is not simple, the coefficient of the third power of LNEXPER might be important for prediction.

We remember that variables whose coefficients are not significant can be kept in the model when the model is built with predictive purposes.

We compare the MSE_{training} of the two models.

```
with(indwages, mean((LNWAGE - predict(output6))^2))
## [1] 0.07684569
with(indwages, mean((LNWAGE - predict(output7))^2))
## [1] 0.07686349
```

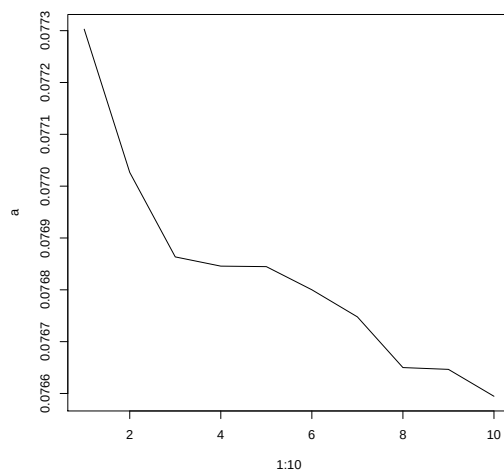
Is it surprising that $MSE_{\text{training}}(\text{output7}) > MSE_{\text{training}}(\text{output6})$? No, why?

4.58

We write a function `model.i` that estimates the model with a i th order polynomial in LNEXPER , and evaluate MSE_{training} , which is decreasing with respect to the order of the polynomial.

```
model.i <- function(i) lm(LNWAGE ~ sex + educf + poly(LNEXPER,
  i), data = indwages)
(n <- nrow(indwages))
a <- sapply(1:10, function(i) {
  with(indwages, mean((LNWAGE - predict(model.i(i)))^2))
})
plot(1:10, a, type = "l")
```

4.59



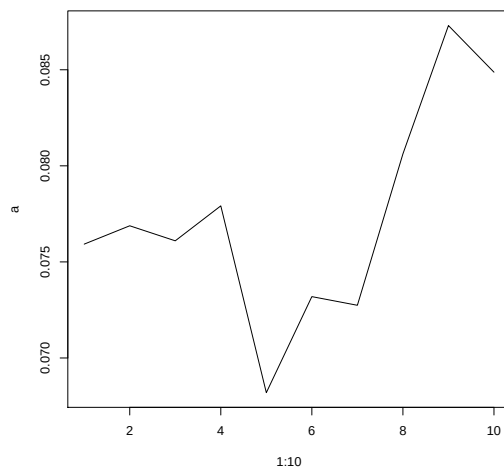
4.60

Therefore, we perform evaluation by having recourse to cross-validation techniques.
Evaluation by means of the validation set approach

```
validation.set <- function(formula) {
  train <- sample(n, n/2)
  val.output5 <- lm(formula, data = indwages, subset = train)
  with(indwages, mean((LNWAGE - predict(val.output5, indwages))[-train]^2))
}

a <- sapply(1:10, function(i) validation.set(LNWAGE ~ sex + educf +
  poly(LNEXPER, i)))
plot(1:10, a, type = "l")
```

4.61



4.62

Observe that evaluation highly depends on which statistical units are in the validation or in the held out set

```
table(replicate(1000, which.min(sapply(1:10, function(i) {
  validation.set(LNWAGE ~ sex + educf + poly(LNEXPER, i))
}))))
##
```

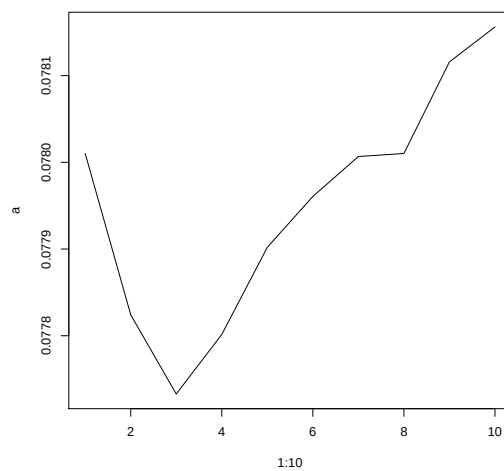
```
##    1    2    3    4    5    6    7    8    9   10
## 116 108 119 114 101  92 103  78  83  86
```

4.63

Evaluation with LOOCV

```
library(boot)
cv.error <- function(i) {
  glm.fit1 <- glm(LNWAGE ~ sex + educf + poly(LNEXPER, i),
    data = indwages)
  cv.glm(indwages, glm.fit1)$delta[1]
}
(a <- sapply(1:10, function(i) cv.error(i)))
plot(1:10, a, type = "l")
```

4.64

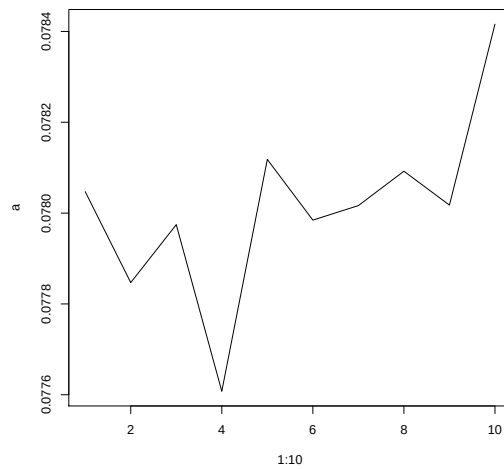


4.65

Evaluation with k -Fold CV

```
library(boot)
cv.error <- function(i) {
  glm.fit1 <- glm(LNWAGE ~ sex + educf + poly(LNEXPER, i),
    data = indwages)
  cv.glm(indwages, glm.fit1, K = 10)$delta[1]
}
(a <- sapply(1:10, function(i) cv.error(i)))
plot(1:10, a, type = "l")
```

4.66



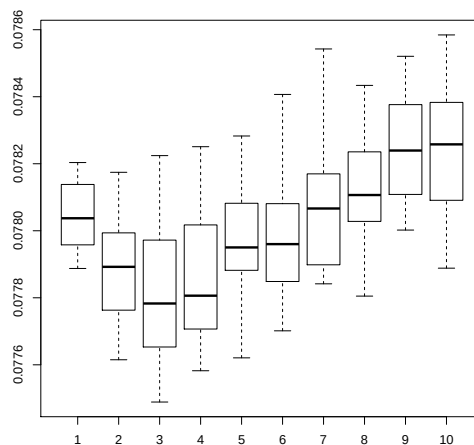
4.67

k -Fold CV gets its minimum when powers of `LNXPER` up to 4 are included in the model. This would be not so a parsimonious model. Moreover, MSE_{test} estimates do not change substantially on varying powers. So one could rely on the solution with powers up to 3. It is a good practice to repeat the procedure of k -Fold CV, in order to assess its stability (when computation is feasible); or, at least, check how the final conclusion depends on the variability of the partial estimates corresponding to the k folds that contribute to the estimation of MSE_{test} .

By replicating 20 times the procedure of k -Fold CV

```
(a <- replicate(20, sapply(1:10, function(i) cv.error(i))))
boxplot(t(a))
```

4.68



4.69

We can observe that the median of the estimates of MSE_{test} is practically equal in models with powers of `LNXPER` up to 2 or 3 or 4. So we conclude to consider the more parsimonious model with `LNXPER` and its square. The model including the linear effect of `LNXPER` only could also be a good choice, by having regard at the reduction of the MSE_{test} estimate.

4.70

6.2 Bootstrap Non-Parametric Evaluation

We now consider an example of inferential model evaluation with bootstrap techniques

```
boot.fn <- function(data, index) lm(LNWAGE ~ sex + educf + poly(LNEXPER,  
  2), data = data, subset = index)$coef
```

4.71

This is equivalent to the initial estimate of the model.

```
boot.fn(indwages, 1:n)
```

4.72

This is one bootstrap replication.

```
set.seed(123)  
boot.fn(indwages, sample(n, n, replace = TRUE))
```

With the function `boot` we can perform an appropriate number of bootstrap replications.

4.73

```
(a <- boot(indwages, boot.fn, 1000))  
##  
## ORDINARY NONPARAMETRIC BOOTSTRAP  
##  
##  
## Call:  
## boot(data = indwages, statistic = boot.fn, R = 1000)  
##  
##  
## Bootstrap Statistics :  
##      original      bias      std. error  
## t1* 1.8854773  0.0005761243  0.02609331  
## t2* 0.1122402  0.0001465592  0.01514578  
## t3* 0.1493657 -0.0007711753  0.02737565  
## t4* 0.3188040 -0.0008605576  0.02675467  
## t5* 0.4860008 -0.0009091868  0.02725503  
## t6* 0.6490098 -0.0010432438  0.03135585  
## t7* 6.5484209  0.0029512083  0.27216922  
## t8* 0.6482043 -0.0136945266  0.26486036
```

4.74

Bootstrap estimates are very close to OLS, but estimated standard errors differs

```
output5 <- lm(LNWAGE ~ sex + educf + poly(LNEXPER, 2), data = indwages)  
coeftest(output5)  
##  
## t test of coefficients:  
##  
##              Estimate Std. Error t value Pr(>|t|)  
## (Intercept)    1.885477   0.030734  61.3488 < 2.2e-16 ***  
## sexm           0.112240   0.015289   7.3413 3.496e-13 ***  
## educf2         0.149366   0.032996   4.5268 6.472e-06 ***  
## educf3         0.318804   0.031858  10.0071 < 2.2e-16 ***  
## educf4         0.486001   0.032926  14.7602 < 2.2e-16 ***  
## educf5         0.649010   0.032993  19.6713 < 2.2e-16 ***  
## poly(LNEXPER, 2)1 6.548421  0.291887  22.4348 < 2.2e-16 ***  
## poly(LNEXPER, 2)2 0.648204  0.282946   2.2909  0.02211 *  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.75

The conclusion is valid also when we have recourse to classical robust standard errors obtained with a heteroskedasticity-consistent estimate of the covariance matrix of coeffi-

cents.

```
library(sandwich)
coeftest(output5, vcov = vcovHC(output5))
##
## t test of coefficients:
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.885477   0.026609  70.8594 < 2.2e-16 ***
## sexm           0.112240   0.015487   7.2472 6.863e-13 ***
## educf2         0.149366   0.029021   5.1467 3.010e-07 ***
## educf3         0.318804   0.027685  11.5153 < 2.2e-16 ***
## educf4         0.486001   0.028036  17.3350 < 2.2e-16 ***
## educf5         0.649010   0.031377  20.6846 < 2.2e-16 ***
## poly(LNEXPER, 2)1 6.548421  0.273419  23.9501 < 2.2e-16 ***
## poly(LNEXPER, 2)2 0.648204  0.269419   2.4059  0.01625 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.76

We can obtain bootstrap confidence intervals for parameter estimates by computing the 0.025 and 0.975 percentiles of the bootstrap results, as well as histograms or kernel density plots of bootstrap estimates

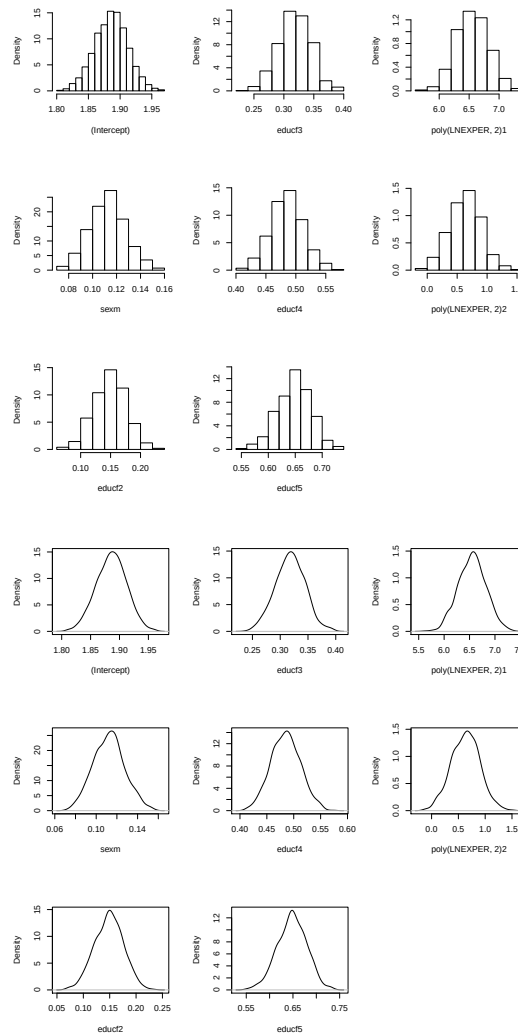
```
anames <- names(coef(output5))
confint <- sapply(1:8, function(i) quantile(a$t[, i], c(0.025,
  0.975)))
colnames(confint) <- anames
round(t(confint), 4)
##              2.5%  97.5%
## (Intercept)    1.8348 1.9364
## sexm           0.0835 0.1425
## educf2         0.0951 0.2012
## educf3         0.2647 0.3701
## educf4         0.4330 0.5402
## educf5         0.5849 0.7066
## poly(LNEXPER, 2)1 6.0267 7.0816
## poly(LNEXPER, 2)2 0.1115 1.1649
```

4.77

```
layout(matrix(1:9, 3, 3))
sapply(1:8, function(i) hist(a$t[, i], main = "", xlab = anames[i],
  freq = FALSE))
```

```
layout(matrix(1:9, 3, 3))
sapply(1:8, function(i) {
  plot(density(a$t[, i]), main = "", xlab = anames[i])
})
```

4.78



4.79

4.80

6.3 Effect Sizes

We conclude the analysis of the data set, by evaluating the partial R^2 and ΔR^2 related to predictors.

We first evaluate partial models

```
output5_drop_sex <- lm(LNWAGE ~ educf + poly(LNEXPER, 2), data = indwages)
output5_drop_educf <- lm(LNWAGE ~ sex + poly(LNEXPER, 2), data = indwages)
output5_drop_LNEXPER <- lm(LNWAGE ~ sex + educf, data = indwages)
```

4.81

In order to evaluate the partial R^2 we extract residuals

```
complete <- output5$res
drop_sex <- output5_drop_sex$res
drop_educf <- output5_drop_educf$res
drop_LNEXPER <- output5_drop_LNEXPER$res
```

4.82

and then compute the proportion of residual variation explained by adding the predictor.

```
pr_sex <- (var(drop_sex) - var(complete))/var(drop_sex)
pr_educf <- (var(drop_educf) - var(complete))/var(drop_educf)
pr_LNEXPER <- (var(drop_LNEXPER) - var(complete))/var(drop_LNEXPER)
```

4.83

In order to evaluate the ΔR^2 we extract the R^2 s

```
complete <- summary(output5)$r.squared
drop_sex <- summary(output5_drop_sex)$r.squared
drop_educf <- summary(output5_drop_educf)$r.squared
drop_LNEXPER <- summary(output5_drop_LNEXPER)$r.squared
```

4.84

and then obtain the ΔR^2 as the increase in R^2 obtained by adding the predictor.

```
a <- c(complete - drop_sex, complete - drop_educf, complete -
      drop_LNEXPER)
names(a) <- c("DeltaR2_sex", "DeltaR2_educf", "DeltaR2_LNEXPER")
a
##      DeltaR2_sex      DeltaR2_educf DeltaR2_LNEXPER
##      0.02185138      0.27520377      0.20466584
```

4.85

With both types of effect measures we can conclude that education and experience have the largest effects in explaining wages.

Effect sizes measures can be obtained from partial squared correlations as $f^2 = \frac{R^2_{\text{partial}}}{1 - R^2_{\text{partial}}}$.

4.86

We obtain

```
a <- c(pr_sex/(1 - pr_sex), pr_educf/(1 - pr_educf), pr_LNEXPER/(1 -
      pr_LNEXPER))
names(a) <- c("Effect.size_sex", "Effect.size_educf", "Effect.size_LNEXPER")
a
##      Effect.size_sex      Effect.size_educf Effect.size_LNEXPER
##      0.03683837      0.46395509      0.34503799
```

As suggested by Cohen, an f^2 of 0.02 is considered to be a small effect, 0.15 a medium effect, and 0.35 a large effect. So we can conclude that effect sizes of education and experience are large while that of sex is small.

4.87

Semipartial correlations ΔR^2 can be also obtained by applying the function `calc.relimp` available in the package `relaimpo`. It appears in the column last, among relative importance metrics. The metric `pmvd` is available, by installing the extended version of this package; `pmvd` can be also used to evaluate relative importance of predictors; see `?calc.relimp` for further info.

4.88

```
library(relaimpo)
calc.relimp(output5, type = c("lmg", "last", "first"))
```

4.89